

Java Backend Interview Series

Chapter 1 · Sub-Chapter 1.5

Collections · Queue, Deque & BlockingQueue

PriorityQueue · ArrayDeque · LinkedBlockingQueue · ArrayBlockingQueue · SynchronousQueue · DelayQueue

Building-style Tutorial · 25+ Q&A · Import Blocks · Producer-Consumer · Heap Internals

1 Queue & Deque Interface Hierarchy

1.1 Introduction — The Problem with Arrays and LinkedLists for FIFO

Before the dedicated Queue interfaces, developers used ArrayList or LinkedList for FIFO queues — but ArrayList.remove(0) is O(n) (shifts all elements), and LinkedList exposes all List methods when you only want queue semantics. Java 5 introduced Queue<E> as a first-class interface with FIFO-specific methods, and Java 6 added Deque<E> for double-ended operations. The java.util.concurrent package added blocking variants critical for producer-consumer patterns.

1.2 Complete Interface Hierarchy

```
import java.util.Queue;
import java.util.Deque;
import java.util.ArrayDeque;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

java.util.Collection<E>

■ ■ ■ java.util.Queue<E> // one-ended: add/offer/poll/peek/remove
■ ■ ■ java.util.Deque<E> // double-ended: addFirst/Last, pollFirst/Last
■ ■ ■ ■ java.util.ArrayDeque<E> // resizable circular array (preferred)
■ ■ ■ ■ java.util.LinkedList<E> // doubly-linked (also implements List)
■ ■ ■ java.util.concurrent.BlockingQueue<E> // add blocking put/take
■ ■ ■ LinkedBlockingQueue<E> // optionally bounded, linked nodes
■ ■ ■ ArrayBlockingQueue<E> // bounded, single lock, array-backed
■ ■ ■ PriorityBlockingQueue<E> // unbounded, priority-ordered
■ ■ ■ DelayQueue<E> // time-delayed entries
■ ■ ■ SynchronousQueue<E> // zero-capacity handoff
■ ■ ■ LinkedTransferQueue<E> // also implements TransferQueue<E>

java.util.concurrent.BlockingDeque<E> // extends BlockingQueue + Deque
■ ■ ■ LinkedBlockingDeque<E>
```

Complete Queue/Deque/BlockingQueue interface hierarchy

1.3 Queue Method Pairs — Throws vs Returns Special Value

Operation	Throws Exception	Returns Special Value
Insert	add(e) — throws IllegalStateException if full	offer(e) — returns false if full
Remove	remove() — throws NoSuchElementException if empty	poll() — returns null if empty

Operation	Throws Exception	Returns Special Value
Examine	element() — throws NoSuchElementException if empty	peek() — returns null if empty
Blocking insert (BlockingQueue)	put(e) — blocks until space available	offer(e, timeout, unit) — waits then false
Blocking remove (BlockingQueue)	take() — blocks until element available	poll(timeout, unit) — waits then null

TIP: Prefer offer/poll/peek over add/remove/element in non-blocking queues. They return null/false instead of throwing exceptions, avoiding try-catch overhead in tight loops.

2 PriorityQueue Deep Dive — Binary Min-Heap

2.1 Introduction — Not a FIFO Queue

PriorityQueue does NOT maintain insertion order. It maintains heap order: the element with the smallest priority (according to natural ordering or Comparator) is always at the head. Internally it uses a binary min-heap stored in a resizable array. Offer/poll are $O(\log n)$; peek is $O(1)$; contains/remove(Object) are $O(n)$ — there is no index structure for arbitrary elements.

2.2 Binary Min-Heap Internals

A binary heap stores a complete binary tree in an array. For element at index i : left child is at $2i+1$, right child at $2i+2$, parent at $(i-1)/2$. The heap property: parent $\leq$ both children (min-heap). offer(e) appends to the end then sifts up (swap with parent while smaller). poll() removes root, moves last element to root, then sifts down.

```
// Heap array visualization:

// Index: [0] [1] [2] [3] [4] [5] [6]

// Value: 1 3 2 7 8 4 6

//

// 1 <- index 0 (root, minimum)

// / \

// 3 2 <- index 1, 2

// / \ / \

// 7 8 4 6 <- index 3,4,5,6

//

// Parent of index i: (i-1) / 2

// Left child of index i: 2*i + 1

// Right child of index i: 2*i + 2

// offer(5): append to index 7, sift up

// [1,3,2,7,8,4,6,5] -> 5 < parent(7) -> [1,3,2,5,8,4,6,7]

// 5 > parent(3) -> done

// poll(): remove root (1), move last to root

// [7,3,2,5,8,4,6] -> sift down: 7 > min(3,2)=2 -> swap with 2

// [2,3,7,5,8,4,6] -> 7 > min(4,6)=4 -> swap with 4

// [2,3,4,5,8,7,6] -> 7 <= min(no children) -> done
```

Binary min-heap: array layout, sift-up (offer), sift-down (poll)

2.3 PriorityQueue — Natural Order and Custom Comparator

```
import java.util.PriorityQueue;
import java.util.Comparator;

// Natural order (min-heap by default for numbers/strings)

PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(5); minHeap.offer(1); minHeap.offer(3);
System.out.println(minHeap.poll()); // 1 (minimum)
System.out.println(minHeap.poll()); // 3
System.out.println(minHeap.poll()); // 5

// Max-heap using reversed comparator

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(5); maxHeap.offer(1); maxHeap.offer(3);
System.out.println(maxHeap.poll()); // 5 (maximum)

// Custom object: sort by deadline (earliest first)
record Task(String name, int deadline) {}

PriorityQueue<Task> taskQueue = new PriorityQueue<>(
    Comparator.comparingInt(Task::deadline)
);
taskQueue.offer(new Task("Deploy", 5));
taskQueue.offer(new Task("Test", 2));
taskQueue.offer(new Task("Review", 8));

while (!taskQueue.isEmpty()) {
    Task t = taskQueue.poll();
    System.out.println(t.name() + " by day " + t.deadline());
}

// Test by day 2
// Deploy by day 5
// Review by day 8
```

PriorityQueue: natural order (min-heap), max-heap, custom Comparator

2.4 PriorityQueue Complexity & Limitations

Operation	Complexity	Notes
offer(e)	O(log n)	Append + sift-up
poll()	O(log n)	Remove root + sift-down
peek()	O(1)	Root is always minimum

Operation	Complexity	Notes
remove(Object)	O(n)	Linear scan to find element, then O(log n) sift
contains(Object)	O(n)	No index structure for arbitrary lookup
Heapify (from collection)	O(n)	PriorityQueue(Collection) uses Floyd's algorithm
Iteration order	Undefined	Iterator does NOT return elements in sorted order

WARNING: PriorityQueue.iterator() does NOT iterate in priority order. To drain in order, use poll() repeatedly or convert to a sorted list via List.copyOf(pq) then Collections.sort().

WARNING: PriorityQueue is NOT thread-safe. For concurrent priority access use PriorityBlockingQueue. Also, PriorityQueue does not support null elements.

3 ArrayDeque Deep Dive — Preferred Stack & Queue

3.1 Introduction — Why Not LinkedList or Stack?

Java's legacy `Stack` class extends `Vector` (fully synchronized — unnecessary overhead). `LinkedList` implements `Deque` but each element allocates a `Node` object (extra memory, poor cache locality). `ArrayDeque` is a resizable circular array that provides $O(1)$ amortized for all operations at both ends, with excellent cache performance. Since Java 6, `ArrayDeque` is the recommended replacement for both `Stack` and `Queue` when thread safety is not needed.

3.2 Circular Array Internals

`ArrayDeque` maintains a fixed-size `Object[]` and two pointers: `head` (index of first element) and `tail` (index past the last element). Adding to the front decrements `head` (wrapping around). Adding to the back increments `tail` (wrapping around). When `head == tail` (array full), the array is doubled and elements are rearranged. This gives $O(1)$ amortized for `addFirst`/`addLast`/`pollFirst`/`pollLast`.

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Queue;

// Internal circular array (simplified)
// Array: [_, _, _, C, A, B, _, _] (capacity 8, wraps around)
// ^ ^tail (exclusive)
// head

// elements: A(index 4), B(index 5), C(index 3) after addFirst(C)

// head = (head - 1) & (elements.length - 1) // addFirst
// tail = (tail + 1) & (elements.length - 1) // addLast
// Both use bitwise AND (power-of-2 capacity) for fast modulo

// ArrayDeque as STACK (LIFO) — preferred over java.util.Stack
Deque<String> stack = new ArrayDeque<>();
stack.push("first"); // same as addFirst()
stack.push("second"); // same as addFirst()
stack.push("third"); // same as addFirst()
System.out.println(stack.pop()); // "third" (LIFO)
System.out.println(stack.peek()); // "second" (no removal)

// ArrayDeque as QUEUE (FIFO) — preferred over LinkedList
Queue<String> queue = new ArrayDeque<>();
queue.offer("first"); // same as addLast()
queue.offer("second");
queue.offer("third");
```

```
System.out.println(queue.poll()); // "first" (FIFO)
```

```
System.out.println(queue.peek()); // "second"
```

ArrayDeque as stack (LIFO) and queue (FIFO)

3.3 Deque Methods — Both Ends

Operation	Front (First)	Back (Last)
Add (throws)	addFirst(e)	addLast(e)
Add (returns false)	offerFirst(e)	offerLast(e)
Remove (throws)	removeFirst()	removeLast()
Remove (returns null)	pollFirst()	pollLast()
Peek (throws)	getFirst()	getLast()
Peek (returns null)	peekFirst()	peekLast()
Stack alias	push(e) = addFirst(e)	—
Stack alias	pop() = removeFirst()	—

TIP: ArrayDeque is faster than LinkedList for both stack and queue use cases in benchmarks. LinkedList allocates a Node object per element; ArrayDeque uses a plain array — much better cache locality.

WARNING: ArrayDeque does NOT allow null elements. Storing null would break peek()/poll() which return null to signal "empty". Also, ArrayDeque is not thread-safe — use LinkedBlockingDeque for concurrent deque operations.

4 LinkedBlockingQueue & ArrayBlockingQueue

4.1 Introduction — The Producer-Consumer Problem

The producer-consumer pattern requires a thread-safe bounded buffer: producers add work items and block when the queue is full; consumers remove items and block when the queue is empty. Java's `BlockingQueue` implementations solve this with `put()/take()` blocking calls and `offer()/poll()` timeout-based calls, eliminating the need for manual synchronization with `wait()/notifyAll()`.

4.2 LinkedBlockingQueue — Optionally Bounded, Two Locks

`LinkedBlockingQueue` uses a linked list of nodes. It has TWO `ReentrantLock`s: a `putLock` for producers and a `takeLock` for consumers. This allows concurrent `put` and `take` operations — producers and consumers never block each other unless the queue is full or empty. Default capacity is `Integer.MAX_VALUE` (effectively unbounded). Bounded with `LinkedBlockingQueue(capacity)`.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;
import java.util.List;
import java.util.ArrayList;

// Unbounded (capacity = Integer.MAX_VALUE)
BlockingQueue<String> unbounded = new LinkedBlockingQueue<>();

// Bounded (capacity = 100)
BlockingQueue<String> bounded = new LinkedBlockingQueue<>(100);

// put() – blocks if full (bounded only)
bounded.put("task"); // blocks thread if queue has 100 elements

// take() – blocks if empty
String task = bounded.take(); // blocks until an element is available

// offer() with timeout – non-blocking with wait
boolean added = bounded.offer("task", 500, TimeUnit.MILLISECONDS);
if (!added) System.out.println("Queue full, task dropped");

// poll() with timeout
String item = bounded.poll(1, TimeUnit.SECONDS);
if (item == null) System.out.println("Queue empty after 1s");

// drainTo() – batch transfer (efficient)
List<String> batch = new ArrayList<>();
int count = bounded.drainTo(batch, 10); // transfer up to 10 elements
```

LinkedBlockingQueue: put/take blocking, offer/poll with timeout, drainTo

4.3 ArrayBlockingQueue — Bounded, Single Lock, Fair Mode

ArrayBlockingQueue is always bounded (capacity set at construction). It uses a single ReentrantLock for both put and take (unlike LinkedBlockingQueue's two locks). This means put and take can block each other. The advantage: optional fairness mode (fair=true) which uses a FIFO ordering of waiting threads via a condition queue, preventing thread starvation. The array is pre-allocated at construction — no GC pressure from node allocation.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

// Bounded, unfair (default) – higher throughput
BlockingQueue<Integer> unfair = new ArrayBlockingQueue<>(10);

// Bounded, fair – FIFO ordering of waiting threads (lower throughput)
BlockingQueue<Integer> fair = new ArrayBlockingQueue<>(10, true);

// ArrayBlockingQueue: no node allocation – array pre-allocated
// Put and take share ONE lock – slightly lower throughput than LinkedBQ
// But: bounded with predictable memory, suitable for back-pressure

// Back-pressure pattern: producer blocks when consumer is slow
BlockingQueue<WorkItem> workQueue = new ArrayBlockingQueue<>(50);

// Producer: naturally rate-limited by queue capacity
// workQueue.put(item) blocks when queue is full – back-pressure!

// Consumer: pulls as fast as it can
// workQueue.take() blocks when queue is empty – no busy waiting
```

ArrayBlockingQueue: bounded, optional fair mode, back-pressure

Feature	LinkedBlockingQueue	ArrayBlockingQueue
Backing structure	Linked nodes	Pre-allocated array
Capacity	Optional (default: unbounded)	Mandatory at construction
Lock	Two locks (put + take)	One lock
Concurrent put+take?	Yes (different locks)	No (same lock)
Fair mode	Not available	Yes (fair=true)
Memory allocation	Node per element (GC)	Fixed, no GC
Throughput	Higher (two locks)	Slightly lower (one lock)
Use case	General producer-consumer	Bounded buffer, back-pressure

5 SynchronousQueue, DelayQueue, PriorityBlockingQueue

5.1 SynchronousQueue — Zero-Capacity Handoff

SynchronousQueue has zero capacity — it does not store elements. Every put() must wait for a corresponding take(), and vice versa. This is a direct handoff between threads. Executors.newCachedThreadPool() uses a SynchronousQueue — if no idle thread is waiting to take(), a new thread is spawned. This makes it ideal for work-stealing and thread-pool handoff patterns.

```
import java.util.concurrent.SynchronousQueue;
import java.util.concurrent.Executors;
import java.util.concurrent.ExecutorService;

SynchronousQueue<String> handoff = new SynchronousQueue<>();

// Producer thread
Thread producer = new Thread(() -> {
    try {
        System.out.println("Producer: waiting for consumer...");
        handoff.put("ITEM"); // BLOCKS until a consumer calls take()
        System.out.println("Producer: handoff complete");
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

// Consumer thread
Thread consumer = new Thread(() -> {
    try {
        Thread.sleep(500); // simulate delay
        String item = handoff.take(); // BLOCKS until producer calls put()
        System.out.println("Consumer: received " + item);
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

producer.start(); consumer.start();

// Output:
// Producer: waiting for consumer...
// Consumer: received ITEM
// Producer: handoff complete

// Executors.newCachedThreadPool() uses SynchronousQueue internally
ExecutorService cached = Executors.newCachedThreadPool();
// put(task) -> if no idle thread -> spawn new thread -> take(task)
```

```
// Creates threads on demand; kills idle threads after 60s
```

SynchronousQueue: zero-capacity, direct thread-to-thread handoff

5.2 DelayQueue — Time-Based Scheduling

DelayQueue holds elements that implement the Delayed interface. An element is only available for take() after its delay has expired. Elements with the shortest remaining delay are at the head. This makes DelayQueue a natural choice for task schedulers, cache TTL expiry, and rate limiting with time windows.

```
import java.util.concurrent.DelayQueue;
import java.util.concurrent.Delayed;
import java.util.concurrent.TimeUnit;

// ScheduledTask implementing Delayed
record ScheduledTask(String name, long fireTimeNanos) implements Delayed {

    // Remaining delay: positive = not yet ready, negative/0 = ready

    @Override
    public long getDelay(TimeUnit unit) {
        return unit.convert(fireTimeNanos - System.nanoTime(), TimeUnit.NANOSECONDS);
    }

    @Override
    public int compareTo(Delayed other) {
        return Long.compare(this.fireTimeNanos,
            ((ScheduledTask)other).fireTimeNanos);
    }
}

// Usage
DelayQueue<ScheduledTask> scheduler = new DelayQueue<>();

long now = System.nanoTime();
scheduler.put(new ScheduledTask("Task A", now + TimeUnit.SECONDS.toNanos(3)));
scheduler.put(new ScheduledTask("Task B", now + TimeUnit.SECONDS.toNanos(1)));
scheduler.put(new ScheduledTask("Task C", now + TimeUnit.SECONDS.toNanos(2)));

// Consumer: blocks until next task is ready
while (!scheduler.isEmpty()) {
    ScheduledTask task = scheduler.take(); // blocks until delay expires
    System.out.println("Running: " + task.name());
}

// Output (in order): Task B (1s), Task C (2s), Task A (3s)
```

DelayQueue with ScheduledTask record — time-based scheduling

5.3 PriorityBlockingQueue — Unbounded Priority Queue

PriorityBlockingQueue is the thread-safe version of PriorityQueue. It is unbounded — put() never blocks (it always succeeds unless out of memory). take() blocks when the queue is empty. Internally it uses the same binary min-heap as PriorityQueue but with a ReentrantLock protecting all operations. Elements must implement Comparable or a Comparator must be provided.

```
import java.util.concurrent.PriorityBlockingQueue;
import java.util.Comparator;

// PriorityBlockingQueue – thread-safe priority queue

PriorityBlockingQueue<Task> queue = new PriorityBlockingQueue<>(
    11,
    Comparator.comparingInt(Task::priority).reversed() // high priority first
);

record Task(String name, int priority) {}

queue.put(new Task("Low priority cleanup", 1));
queue.put(new Task("Critical payment", 10));
queue.put(new Task("Normal processing", 5));

// Consumer thread

Task next = queue.take(); // blocks if empty; returns highest priority
System.out.println(next.name()); // Critical payment (priority 10)

// WARNING: put() NEVER blocks (unbounded)

// Memory can grow without bound under sustained producer load

// Use ArrayBlockingQueue or LinkedBlockingQueue for back-pressure
```

PriorityBlockingQueue — thread-safe, unbounded, priority-ordered

WARNING: PriorityBlockingQueue is unbounded. Producers can enqueue indefinitely without blocking, leading to OutOfMemoryError under sustained load. Always ensure consumers are keeping up, or use a bounded queue for back-pressure.

6 Producer-Consumer Pattern — Classic Implementation

6.1 Introduction — The Canonical Concurrent Pattern

Producer-consumer decouples work generation from work processing. Producers add items to a shared buffer; consumers remove and process them. The buffer (BlockingQueue) handles synchronization automatically — producers block when full (back-pressure), consumers block when empty. This eliminates the need for manual synchronized/wait/notifyAll code.

6.2 Classic Implementation — Before (Manual sync) vs After (BlockingQueue)

```
import java.util.LinkedList;

// BEFORE – manual synchronized + wait/notifyAll (error-prone)

class ManualBuffer<T> {

    private final LinkedList<T> queue = new LinkedList<>();

    private final int capacity;

    synchronized void put(T item) throws InterruptedException {

        while (queue.size() == capacity) wait(); // busy loop

        queue.add(item);

        notifyAll(); // wake all waiting threads

    }

    synchronized T take() throws InterruptedException {

        while (queue.isEmpty()) wait();

        T item = queue.removeFirst();

        notifyAll();

        return item;

    }

}
```

BEFORE: Manual producer-consumer with synchronized + wait/notifyAll

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.List;
import java.util.ArrayList;

// AFTER – clean, readable, correct with ArrayBlockingQueue

public class ProducerConsumerDemo {

    private static final int CAPACITY = 10;

    private static final int POISON_PILL_COUNT = 3; // one per consumer

    public static void main(String[] args) throws InterruptedException {
```

```

BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(CAPACITY);

// Producer

Thread producer = new Thread(() -> {
    try {
        for (int i = 1; i <= 20; i++) {
            queue.put(i); // blocks if queue is full

            System.out.println("Produced: " + i);
        }

        // Send poison pills to stop consumers

        for (int i = 0; i < POISON_PILL_COUNT; i++) {
            queue.put(-1); // -1 = shutdown signal
        }

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

// Consumers (3 threads)

List<Thread> consumers = new ArrayList<>();

for (int c = 0; c < 3; c++) {
    final int id = c + 1;

    Thread consumer = new Thread(() -> {
        try {
            while (true) {
                int item = queue.take(); // blocks if empty

                if (item == -1) break; // poison pill

                System.out.println("Consumer-" + id + " processed: " + item);
            }

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });

    consumers.add(consumer);
}

producer.start();

consumers.forEach(Thread::start);

```

```

producer.join();

consumers.forEach(t -> { try { t.join(); } catch (Exception e) {} });

}

}

```

AFTER: ArrayBlockingQueue producer-consumer — clean and correct

TIP: The poison pill pattern (a sentinel value like -1 or a special object) is the standard way to signal consumers to stop. Send one poison pill per consumer thread to ensure all consumers see the shutdown signal.

6.3 ExecutorService + BlockingQueue Pattern

```

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.TimeUnit;

// Production-grade: ExecutorService manages consumer threads
BlockingQueue<Runnable> taskQueue = new LinkedBlockingQueue<>(100);
ExecutorService executor = new ThreadPoolExecutor(
    4, // corePoolSize
    4, // maximumPoolSize
    0L, TimeUnit.MILLISECONDS,
    taskQueue, // the BlockingQueue IS the work queue
    new ThreadPoolExecutor.CallerRunsPolicy() // back-pressure: caller runs
);

// Submit tasks (producer)
for (int i = 0; i < 50; i++) {
    final int taskId = i;
    executor.submit(() -> {
        System.out.println("Processing task " + taskId
            + " on " + Thread.currentThread().getName());
    });
}

// When queue is full + all threads busy + CallerRunsPolicy:
// The CALLING thread runs the task (natural back-pressure)

executor.shutdown();
executor.awaitTermination(30, TimeUnit.SECONDS);

```

ThreadPoolExecutor with bounded BlockingQueue for back-pressure

7 Deque as Stack vs Queue — Method Mapping

7.1 Introduction

Deque (Double-Ended Queue) is the most flexible sequential data structure in Java. When you restrict to front-only operations it becomes a stack; when you restrict to back-insert + front-remove it becomes a FIFO queue. `ArrayDeque` should be the default choice for both use cases.

7.2 Deque Stack vs Queue Method Mapping

Action	Stack (LIFO) method	Queue (FIFO) method	Deque unified method
Add element	<code>push(e)</code> / <code>addFirst(e)</code>	<code>offer(e)</code> / <code>addLast(e)</code>	<code>offerFirst</code> / <code>offerLast</code>
Remove & return	<code>pop()</code> / <code>removeFirst()</code>	<code>poll()</code> / <code>removeFirst()</code>	<code>pollFirst</code> / <code>pollLast</code>
Peek (no remove)	<code>peek()</code> / <code>peekFirst()</code>	<code>peek()</code> / <code>peekFirst()</code>	<code>peekFirst</code> / <code>peekLast</code>
Check empty	<code>isEmpty()</code>	<code>isEmpty()</code>	<code>isEmpty()</code>

```
import java.util.ArrayDeque;
import java.util.Deque;

// Balanced parentheses checker using Deque as Stack

public boolean isBalanced(String expr) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : expr.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false;
        }
    }
    return stack.isEmpty();
}

// BFS using Deque as Queue

public void bfs(Node root) {
    Deque<Node> queue = new ArrayDeque<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
```

```
Node node = queue.poll();

System.out.println(node.val);

if (node.left != null) queue.offer(node.left);

if (node.right != null) queue.offer(node.right);

}

}
```

Deque as stack (balanced parens) and queue (BFS)

8 Performance & Thread Safety Comparison

8.1 Comprehensive Comparison Table

Implementation	Ordering	Bounded?	Thread Safe	put blocks?	take blocks?	Null?	Primary Use Case
PriorityQueue	Priority (min-heap)	No	No	N/A	N/A	No	Single-thread scheduling
ArrayDeque	FIFO or LIFO	No	No	N/A	N/A	No	Stack/Queue (single thread)
LinkedList	FIFO or LIFO	No	No	N/A	N/A	Yes	Legacy deque (avoid)
ArrayBlockingQueue	FIFO	Yes	Yes	Yes	Yes	No	Bounded producer-consumer
LinkedBlockingQueue	FIFO	Optional	Yes	If bounded	Yes	No	General producer-consumer
PriorityBlockingQueue	Priority	No (unbounded)	Yes	No	Yes	No	Concurrent scheduling
SynchronousQueue	N/A (zero cap)	Zero cap	Yes	Yes (always)	Yes (always)	No	Thread handoff
DelayQueue	Delay order	No	Yes	No	Yes (until ready)	No	Timed task scheduling
LinkedTransferQueue	FIFO	No	Yes	On transfer()	Yes	No	High-perf handoff

8.2 Performance Characteristics

Operation	PriorityQueue	ArrayDeque	LinkedBlockingQueue	ArrayBlockingQueue
offer/add	$O(\log n)$	$O(1)$ amortized	$O(1)$	$O(1)$
poll/remove	$O(\log n)$	$O(1)$ amortized	$O(1)$	$O(1)$
peek/element	$O(1)$	$O(1)$	$O(1)$	$O(1)$
contains	$O(n)$	$O(n)$	$O(n)$	$O(n)$
size	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Memory	Object[] array	Object[] array	Node per element	Pre-alloc array
GC pressure	Low (resize only)	Low (resize only)	High (node/element)	None (pre-alloc)

TIP: For maximum producer-consumer throughput with two locks: `LinkedBlockingQueue`. For predictable bounded memory and back-pressure: `ArrayBlockingQueue`. For priority-based work: `PriorityBlockingQueue`. For direct thread handoff: `SynchronousQueue`.

9 LinkedTransferQueue & Concurrent Deque Patterns

9.1 LinkedTransferQueue — High-Performance Handoff

`LinkedTransferQueue` (Java 7+) implements `TransferQueue<E>`, which extends `BlockingQueue<E>`. The key addition is `transfer(e)`: like `SynchronousQueue.put()` but only for that one element — if a consumer is waiting, handoff is direct; otherwise the element is enqueued. This allows mixing buffered and direct-handoff semantics in the same queue.

```
import java.util.concurrent.LinkedTransferQueue;
import java.util.concurrent.TimeUnit;

LinkedTransferQueue<String> queue = new LinkedTransferQueue<>();

// transfer(e): blocks until a consumer takes e directly
// (bypasses the queue if a thread is already waiting)
Thread producer = new Thread(() -> {
    try {
        queue.transfer("URGENT"); // direct handoff if consumer waiting
        System.out.println("URGENT delivered");
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

// tryTransfer(e): non-blocking; returns false if no consumer waiting
boolean handed = queue.tryTransfer("LOW_PRIORITY");
if (!handed) queue.put("LOW_PRIORITY"); // fallback to enqueue

// tryTransfer(e, timeout, unit): timed version
boolean transferred = queue.tryTransfer("ITEM", 500, TimeUnit.MILLISECONDS);

// hasWaitingConsumer(): check if any thread is blocked in take()
if (queue.hasWaitingConsumer()) {
    queue.transfer("HIGH_PRIORITY"); // guaranteed direct handoff
}
```

LinkedTransferQueue: transfer(), tryTransfer(), hasWaitingConsumer()

9.2 LinkedBlockingDeque — Concurrent Double-Ended Queue

`LinkedBlockingDeque` implements both `BlockingQueue` and `Deque`. It supports blocking operations at both ends. Key use case: work-stealing thread pools where each thread has a local deque and steals from the tail of other threads' deques when its own is empty. This reduces contention compared to a shared central queue.

```
import java.util.concurrent.LinkedBlockingDeque;
import java.util.List;
import java.util.concurrent.TimeUnit;
```

```

LinkedBlockingDeque<String> deque = new LinkedBlockingDeque<>(100);

// Front operations (LIFO - stack-like for work stealing)
deque.putFirst("task1"); // blocking add to front
deque.putLast("task2"); // blocking add to back

String own = deque.takeFirst(); // own thread takes from front (LIFO)
String steal = deque.takeLast(); // stealing thread takes from back

// Work-stealing pattern
class WorkStealingPool {
    private final List<LinkedBlockingDeque<Runnable>> queues;
    private final int nThreads;

    void workerLoop(int id) throws InterruptedException {
        LinkedBlockingDeque<Runnable> myQueue = queues.get(id);
        while (true) {
            Runnable task = myQueue.pollFirst(10, TimeUnit.MILLISECONDS);
            if (task == null) {
                // steal from another thread's deque
                int victim = (id + 1) % nThreads;
                task = queues.get(victim).pollLast(); // steal from tail
            }
            if (task != null) task.run();
        }
    }
}

```

LinkedBlockingDeque: blocking deque, work-stealing pattern

TIP: Java's ForkJoinPool uses a similar work-stealing deque internally. For most use cases, prefer ForkJoinPool over building your own work-stealing pool with LinkedBlockingDeque.

11 Queue in Algorithm Problems — Interview Patterns

11.1 Introduction

Queues and deques appear in a wide range of algorithm interview problems. The most common patterns: BFS level-order traversal, monotonic deque for sliding window problems, TopK with PriorityQueue, and task scheduling simulations. Mastering these patterns with the right Queue implementation is key to cracking interviews at top companies.

11.2 BFS Level-Order Traversal with Queue

```
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.List;
import java.util.ArrayList;

// Binary tree level-order traversal using ArrayDeque
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size(); // snapshot: all nodes at this level
        List<Integer> level = new ArrayList<>(levelSize);

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}

// For input tree:
// 3
// / \
```

```
// 9 20
// / \
// 15 7

// Result: [[3], [9,20], [15,7]]
```

BFS level-order traversal using ArrayDeque Queue

11.3 Top K Elements with PriorityQueue

```
import java.util.PriorityQueue;
import java.util.HashMap;
import java.util.Map;
import java.util.Comparator;

// Find top K frequent elements using min-heap of size K
public static int[] topKFrequent(int[] nums, int k) {
    // Step 1: count frequencies
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    // Step 2: min-heap of size k (removes least frequent when overflow)
    PriorityQueue<Map.Entry<Integer,Integer>> heap =
    new PriorityQueue<>(Comparator.comparingInt(Map.Entry::getValue));

    for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll(); // remove least frequent
    }

    // Step 3: extract remaining k elements
    int[] result = new int[k];
    for (int i = k-1; i >= 0; i--) {
        result[i] = heap.poll().getKey();
    }
    return result;
}

// Test: topKFrequent([1,1,1,2,2,3], 2) -> [1, 2]
// Why min-heap of size k? Keep the k LARGEST frequencies.
// A smaller heap element means less frequent -> poll() removes it.
// O(n log k) time, O(n) space
```

Top K frequent elements — min-heap of size K pattern

11.4 Monotonic Queue — Next Greater Element

```

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Arrays;

// Next Greater Element: for each element, find the next larger one
// O(n) using a monotonic decreasing stack (implemented with Deque)
public static int[] nextGreater(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);

    Deque<Integer> stack = new ArrayDeque<>(); // indices

    for (int i = 0; i < n; i++) {
        // While stack not empty AND current > top of stack
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
            int idx = stack.pop();
            result[idx] = nums[i]; // nums[i] is next greater for idx
        }
        stack.push(i);
    }
    return result;
}

// Test: nextGreater([2,1,2,4,3]) -> [4,2,4,-1,-1]
// Trace:
// i=0: push 0. stack=[0]
// i=1: 1<2, push 1. stack=[1,0]
// i=2: 2>1 -> pop 1, result[1]=2; 2==2, push 2. stack=[2,0]
// i=3: 4>2 -> pop 2, result[2]=4; 4>2 -> pop 0, result[0]=4. stack=[3]
// i=4: 3<4, push 4. stack=[4,3]
// Remaining: result[3]=result[4]=-1

```

Next Greater Element — monotonic stack with ArrayDeque

TIP: The monotonic stack pattern (decreasing or increasing) solves Next Greater, Largest Rectangle in Histogram, Trapping Rain Water, and Daily Temperatures in O(n). ArrayDeque is the preferred deque implementation over Stack or LinkedList.

12 Threading Deep Dive — Lock Analysis & Memory Model

12.1 ArrayBlockingQueue Lock Internals

`ArrayBlockingQueue` uses a single `ReentrantLock` with two `Conditions`: `notEmpty` (consumers wait here when queue is empty) and `notFull` (producers wait here when queue is full). When a producer calls `put(e)`, it acquires the lock, checks `count < items.length`, adds the item, increments `count`, and signals `notEmpty`. This wakes one blocked consumer. The consumer acquires the lock, removes the item, decrements `count`, signals `notFull` (waking one producer), and releases the lock.

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.Condition;

// ArrayBlockingQueue lock structure (simplified from JDK)

final ReentrantLock lock;

final Condition notEmpty = lock.newCondition();
final Condition notFull = lock.newCondition();

void put(E e) throws InterruptedException {
    lock.lockInterruptibly();

    try {
        while (count == items.length) notFull.await(); // block if full
        enqueue(e);
    } finally {
        lock.unlock();
    }
}

E take() throws InterruptedException {
    lock.lockInterruptibly();

    try {
        while (count == 0) notEmpty.await(); // block if empty
        return dequeue();
    } finally {
        lock.unlock();
    }
}

private void enqueue(E e) {
    items[putIndex] = e;
```

```

if (++putIndex == items.length) putIndex = 0; // circular
count++;
notEmpty.signal(); // wake one consumer
}

```

ArrayBlockingQueue internal lock structure (simplified)

12.2 LinkedBlockingQueue Two-Lock Protocol

LinkedBlockingQueue uses TWO ReentrantLocks: putLock and takeLock. This allows a producer and a consumer to run simultaneously (as long as neither the full nor empty condition is triggered). The count is an AtomicInteger, updated by both put and take threads without locking both locks at once.

```

import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.atomic.AtomicInteger;

// LinkedBlockingQueue two-lock structure (simplified)

final ReentrantLock putLock = new ReentrantLock();
final ReentrantLock takeLock = new ReentrantLock();

final Condition notFull = putLock.newCondition();
final Condition notEmpty = takeLock.newCondition();
final AtomicInteger count = new AtomicInteger();

void put(E e) throws InterruptedException {
    int c;
    putLock.lockInterruptibly();
    try {
        while (count.get() == capacity) notFull.await();
        enqueue(new Node<>(e));
        c = count.getAndIncrement(); // atomic increment
        if (c + 1 < capacity) notFull.signal(); // more space
    } finally { putLock.unlock(); }
    if (c == 0) signalNotEmpty(); // signal consumer (was empty)
}

// KEY INSIGHT: put() holds putLock; take() holds takeLock
// They can run CONCURRENTLY as long as count > 0 and count < capacity
// This is why LinkedBlockingQueue has higher throughput than ArrayBlockingQueue

```

LinkedBlockingQueue two-lock protocol (simplified)

Property	ArrayBlockingQueue	LinkedBlockingQueue
Locks	1 (put + take share lock)	2 (putLock + takeLock)
Concurrent put+take	No (same lock)	Yes (different locks)

Property	ArrayBlockingQueue	LinkedBlockingQueue
Memory	Pre-allocated array	Node per element (GC)
Capacity	Fixed at construction	Optional (default: MAX_INT)
Fair mode	Yes (fair=true)	No
Throughput	Lower (lock contention)	Higher (two locks)
Predictable latency	Yes (no GC)	No (GC from Node alloc)

10 Coding Exercises

10.1 Exercise A — Sliding Window Max with ArrayDeque

Find the maximum in every sliding window of size k in an array. Use `ArrayDeque` as a monotonic deque to achieve $O(n)$ time complexity.

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Arrays;

// Sliding window maximum —  $O(n)$  with monotonic deque
public static int[] slidingWindowMax(int[] nums, int k) {
    int[] result = new int[nums.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // stores indices

    for (int i = 0; i < nums.length; i++) {
        // Remove indices outside the window
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) {
            dq.pollFirst();
        }

        // Remove smaller elements from back (maintain decreasing order)
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
            dq.pollLast();
        }
        dq.offerLast(i);

        // Front of deque is always the index of the max in current window
        if (i >= k - 1) {
            result[i - k + 1] = nums[dq.peekFirst()];
        }
    }
    return result;
}

// Test
System.out.println(Arrays.toString(
    slidingWindowMax(new int[]{1,3,-1,-3,5,3,6,7}, 3)));
// [3, 3, 5, 5, 6, 7]
```

Exercise A: sliding window max with monotonic ArrayDeque

10.2 Exercise B — Task Scheduler with PriorityQueue

Implement a simple task scheduler that processes tasks in priority order (higher priority first), with ties broken by submission order.

```
import java.util.PriorityQueue;
import java.util.Comparator;

// Exercise B – Priority task scheduler

record Task(String name, int priority, long submitTime) {}

PriorityQueue<Task> scheduler = new PriorityQueue<>(
    Comparator.comparingInt(Task::priority).reversed() // high priority first
    .thenComparingLong(Task::submitTime) // then FIFO
);

long t = System.nanoTime();
scheduler.offer(new Task("Background backup", 1, t+3));
scheduler.offer(new Task("Critical payment", 10, t+1));
scheduler.offer(new Task("Email notification", 5, t+2));
scheduler.offer(new Task("User request", 5, t+0)); // same prio, earlier

while (!scheduler.isEmpty()) {
    Task t2 = scheduler.poll();
    System.out.println("[P" + t2.priority() + "] " + t2.name());
}

// [P10] Critical payment
// [P5] User request (submitted earlier, same priority)
// [P5] Email notification
// [P1] Background backup
```

Exercise B: task scheduler with PriorityQueue + Comparator chaining

10.3 Exercise C — Bounded Producer-Consumer with Back-Pressure

Implement a bounded producer-consumer where the producer naturally slows down when the consumer is slow (back-pressure). Demonstrate that producer blocks when queue is full.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.atomic.AtomicInteger;

// Exercise C – bounded queue demonstrating back-pressure

BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(3); // tiny buffer

AtomicInteger produced = new AtomicInteger();
AtomicInteger consumed = new AtomicInteger();

Thread producer = new Thread(() -> {
```

```

for (int i = 1; i <= 10; i++) {
    try {
        queue.put(i); // BLOCKS when queue has 3 items

        System.out.printf("Produced %d (queue size: %d)%n",
            i, queue.size());

        produced.incrementAndGet();
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}

}, "Producer");

Thread consumer = new Thread(() -> {
    for (int i = 0; i < 10; i++) {
        try {
            Thread.sleep(200); // slow consumer (200ms per item)

            int item = queue.take();

            System.out.printf("Consumed %d%n", item);

            consumed.incrementAndGet();
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

}, "Consumer");

producer.start(); consumer.start();

producer.join(); consumer.join();

System.out.printf("Produced: %d, Consumed: %d%n",
    produced.get(), consumed.get()); // both 10

```

Exercise C: back-pressure with ArrayBlockingQueue(3)

10.4 Exercise D — DelayQueue Rate Limiter

Implement a rate limiter using DelayQueue that allows N requests per second. Each token has a 1/N second delay from the previous token.

```

import java.util.concurrent.DelayQueue;
import java.util.concurrent.Delayed;
import java.util.concurrent.TimeUnit;

// Exercise D – Token bucket rate limiter with DelayQueue

class RateLimiter {

    private final DelayQueue<Token> tokens = new DelayQueue<>();

    record Token(long readyAt) implements Delayed {

        @Override

```

```

public long getDelay(TimeUnit unit) {
    return unit.convert(readyAt - System.nanoTime(), TimeUnit.NANOSECONDS);
}

@Override
public int compareTo(Delayed o) {
    return Long.compare(readyAt, ((Token)o).readyAt());
}
}

public RateLimiter(int ratePerSecond) {
    long interval = TimeUnit.SECONDS.toNanos(1) / ratePerSecond;
    long now = System.nanoTime();
    for (int i = 0; i < ratePerSecond; i++) {
        tokens.put(new Token(now + i * interval));
    }
}

public void acquire() throws InterruptedException {
    Token t = tokens.take(); // blocks until token is ready
    // Refill: next token ready 1 second after this one
    tokens.put(new Token(t.readyAt() + TimeUnit.SECONDS.toNanos(1)));
}

// Usage: 5 requests per second
RateLimiter limiter = new RateLimiter(5);
for (int i = 0; i < 15; i++) {
    limiter.acquire();
    System.out.println("Request " + i + " at " + System.currentTimeMillis());
}

```

Exercise D: token bucket rate limiter using DelayQueue

Cheat Sheet Quick Reference — Queues, Methods & Patterns

CS.1 BlockingQueue Method Cheat Sheet

Method	Behavior	Returns	Throws
add(e)	Insert; full: throws	true	IllegalStateException
offer(e)	Insert; full: false	boolean	—
put(e)	Insert; full: BLOCKS	void	InterruptedException
offer(e,t,u)	Insert; wait t; false	boolean	InterruptedException
remove()	Remove; empty: throws	E	NoSuchElementException
poll()	Remove; empty: null	E or null	—
take()	Remove; empty: BLOCKS	E	InterruptedException
poll(t,u)	Remove; wait t; null	E or null	InterruptedException
peek()	Head; empty: null	E or null	—
element()	Head; empty: throws	E	NoSuchElementException
drainTo(col,n)	Batch transfer up to n	int count	—

CS.2 Queue/Deque Class Selection Guide

Requirement	Best Class	Why
Single-thread FIFO queue	ArrayDeque	O(1) all ops, no GC pressure
Single-thread LIFO stack	ArrayDeque	push()/pop() aliases
Single-thread priority	PriorityQueue	Binary min-heap, O(log n)
Bounded producer-consumer	ArrayBlockingQueue	Back-pressure, pre-alloc memory
Unbounded producer-consumer	LinkedBlockingQueue	Two locks, higher throughput
Thread-safe priority	PriorityBlockingQueue	ReentrantLock wrapped min-heap
Direct thread handoff	SynchronousQueue	Zero-capacity, synchronous
Timed task scheduling	DelayQueue	Delayed interface, sorted by time
Work stealing	LinkedBlockingDeque	takeFirst/takeLast from two ends
Mixed handoff+buffer	LinkedTransferQueue	transfer() + buffered offer()

CS.3 PriorityQueue Internal Structure Reference

Property	Value
Backing store	Object[] (resizable array)
Heap type	Binary min-heap (parent <= children)
Initial capacity	11 (default)
Growth factor	~1.5x if capacity < 64; +50% otherwise

Property	Value
offer() complexity	$O(\log n)$ — sift-up
poll() complexity	$O(\log n)$ — sift-down
peek() complexity	$O(1)$ — root is always minimum
contains() complexity	$O(n)$ — linear scan
remove(Object) complexity	$O(n)$ find + $O(\log n)$ sift = $O(n)$
Heapify from Collection	$O(n)$ — Floyd's build-heap algorithm
Thread safety	NOT thread-safe — use <code>PriorityBlockingQueue</code>
Null elements	Not allowed — throws <code>NullPointerException</code>

CS.4 Memory Model Guarantee for BlockingQueue

Java Memory Model (JMM) §17.4.4: "Actions in a thread prior to placing an object into any concurrent collection happen-before actions subsequent to the access or removal of that element from the collection in another thread." This means: all writes before `put(e)` in Thread A are visible to Thread B after `take()` returns `e`. No additional `volatile` or `synchronized` is needed for the payload itself.

TIP: The JMM happens-before guarantee through `BlockingQueue.put()/take()` means you can safely pass mutable objects between threads WITHOUT synchronizing the fields of those objects, as long as you don't share them after handing off.

Q&A Interview Questions — 25+ Standard & Advanced

Q1-Q12 Standard Questions

Q1. What is the difference between Queue and Deque?

Queue is single-ended: elements are added at the tail and removed from the head (FIFO). Deque is double-ended: elements can be added or removed from both head and tail, supporting both FIFO and LIFO. Deque extends Queue; `ArrayDeque` implements Deque.

Q2. What is the difference between offer() and add() in Queue?

`add(e)` throws `IllegalStateException` if the queue is full (for bounded queues). `offer(e)` returns `false` if the queue is full. Prefer `offer/poll/peek` in non-blocking queues to avoid exception handling overhead.

Q3. What is the internal data structure of PriorityQueue?

A binary min-heap stored in a resizable `Object[]` array. Parent at $(i-1)/2$; left child at $2i+1$; right child at $2i+2$. Heap property: parent $\leq$ both children. `offer()` is $O(\log n)$ sift-up; `poll()` is $O(\log n)$ sift-down; `peek()` is $O(1)$ (root).

Q4. Why is PriorityQueue NOT a sorted queue?

`PriorityQueue` only guarantees that `poll()` returns the minimum element. The internal array does NOT store elements in fully sorted order. `iterator()` returns elements in an arbitrary order. To drain in sorted order, call `poll()` repeatedly.

Q5. Why should you prefer ArrayDeque over Stack or LinkedList?

`java.util.Stack` extends `Vector` which synchronizes every method unnecessarily. `LinkedList` allocates a `Node` per element (poor cache locality). `ArrayDeque` is a resizable circular array with $O(1)$ amortized for all deque ops, no

synchronization, and better memory layout.

Q6. What does put() do in a bounded BlockingQueue when the queue is full?

put() blocks the calling thread indefinitely until space becomes available (a consumer calls take() or poll()). This is the back-pressure mechanism. offer(e, timeout, unit) is the timed variant that returns false if space doesn't open within the timeout.

Q7. What is SynchronousQueue used for?

SynchronousQueue is a zero-capacity handoff queue. Every put() must be matched by a take() before the put() returns, and vice versa. Executors.newCachedThreadPool() uses it: if no idle thread is waiting for take(), a new thread is spawned.

Q8. What interface must elements in DelayQueue implement?

java.util.concurrent.Delayed, which extends Comparable<Delayed>. The getDelay(TimeUnit unit) method returns the remaining delay. An element is only available for take() when getDelay() returns <= 0. Elements are ordered by remaining delay (shortest first).

Q9. What is the difference between LinkedBlockingQueue and ArrayBlockingQueue?

LinkedBlockingQueue: linked nodes (GC per element), optional capacity, TWO locks (put + take can proceed concurrently). ArrayBlockingQueue: pre-allocated array, mandatory bounded capacity, ONE lock (put and take contend), optional fair mode.

Q10. How does PriorityBlockingQueue differ from PriorityQueue?

PriorityBlockingQueue is thread-safe (ReentrantLock), unbounded (put() never blocks), and take() blocks when empty. PriorityQueue is not thread-safe, resizable, and throws NoSuchElementException when empty. Both use binary min-heap internally.

Q11. What is the poison pill pattern?

A sentinel value placed in a BlockingQueue by a producer to signal consumers to stop. Each consumer thread reads the poison pill and exits. The producer must enqueue one poison pill per consumer thread to ensure all consumers see the shutdown signal.

Q12. What is the difference between take() and poll() in BlockingQueue?

take() blocks indefinitely until an element is available. poll() returns null immediately if the queue is empty. poll(timeout, unit) waits up to the given duration then returns null. Use take() for persistent consumers; poll(timeout) for consumers that need to check shutdown periodically.

Q13-Q27 Advanced Questions

Q13. Why does ArrayDeque not allow null elements?

ArrayDeque uses null as a sentinel to indicate an empty slot in the circular array. If null elements were stored, ArrayDeque could not distinguish between an empty slot and a stored null element. This would corrupt the head/tail tracking.

Q14. How does LinkedBlockingQueue achieve concurrent put and take?

It uses TWO ReentrantLocks: putLock (for offer/put) and takeLock (for poll/take). Plus two Conditions: notEmpty (signaled after put; awaited by take when empty) and notFull (signaled after take; awaited by put when full). Since

locks are separate, a producer and consumer can operate simultaneously.

Q15. What is the time complexity of `remove(Object)` in `PriorityQueue`?

$O(n)$ for finding the element (linear scan) + $O(\log n)$ for restoring heap order after removal = $O(n)$ overall. `PriorityQueue` has no index for arbitrary objects — it only guarantees efficient access to the minimum element.

Q16. How does `Executors.newCachedThreadPool()` use `SynchronousQueue`?

The thread pool's work queue is a `SynchronousQueue`. When `execute(task)` is called, it tries to `offer(task)`. If a thread is already waiting in `take()`, the handoff succeeds immediately. If not (no idle thread), `offer()` returns false and the pool creates a new thread that then `take()`s the task.

Q17. What happens if you call `size()` on a `LinkedBlockingQueue` under concurrent modification?

`size()` is accurate (maintained with an `AtomicInteger`) but by the time you use the value another thread may have changed it. It's a point-in-time snapshot. This is fine for monitoring but not for logic like "if `size > threshold`, skip put" — use `offer()` with timeout for that.

Q18. What is `LinkedTransferQueue` and how does it differ from `LinkedBlockingQueue`?

`LinkedTransferQueue` additionally implements `TransferQueue` with `transfer(e)`: the producer blocks until a consumer directly takes the element (like `SynchronousQueue` for that specific element), while other elements in the queue are unaffected. Useful for priority handoff mixed with buffered items.

Q19. Explain binary heap heapification — how does `PriorityQueue(Collection)` build a heap in $O(n)$?

Floyd's algorithm: start from the last non-leaf ($n/2-1$) and sift-down each node toward the root. Each sift-down is $O(\log n)$ but most nodes are near the leaves where sifting is cheap. The mathematical sum telescopes to $O(n)$ total. This is faster than n insertions ($O(n \log n)$).

Q20. What is the fair mode in `ArrayBlockingQueue` and when should you use it?

`fair=true` makes waiting threads served in FIFO order (longest-waiting first) using a fair `ReentrantLock`. This prevents starvation but reduces throughput because all waiting threads must be served in order. Use `fair=true` when individual thread latency matters more than aggregate throughput.

Q21. How do you implement a bounded blocking priority queue?

`PriorityBlockingQueue` is unbounded. To bound it: use a `Semaphore` with initial count = capacity. Producers `acquire()` the semaphore before `put()` (blocking when full). Consumers `release()` after `take()`. This wraps `PriorityBlockingQueue` with bounded semantics.

Q22. What is the difference between `ArrayBlockingQueue` with `fair=false` vs `fair=true`?

`fair=false` (default): threads are woken up non-deterministically — could be any waiting thread. Higher throughput but potential starvation. `fair=true`: strict FIFO ordering of waiting threads via `ReentrantLock(true)`. Lower throughput but guaranteed forward progress for every thread.

Q23. How would you implement a rate limiter using `DelayQueue`?

Maintain a `DelayQueue` of "permit" tokens. Each token has a delay based on the rate (e.g., for 10 req/s: each token fires 100ms after the previous). Requesters call `queue.take()` to acquire a permit — they block if no token is ready. This naturally throttles the request rate.

Q24. What is the memory model guarantee for BlockingQueue operations?

Java Memory Model guarantees that actions in a thread before a `put(e)` happen-before the corresponding `take()` in another thread (synchronizes-before relationship). This means any writes made before `put()` are visible to the thread that does `take()` — no additional volatile/synchronized needed.

Q25. When would you choose BlockingDeque (LinkedBlockingDeque) over BlockingQueue?

When you need work-stealing: multiple consumers each have a local deque; when their local queue is empty, they steal from the tail of another consumer's deque. This reduces contention. `ForkJoinPool` uses a similar work-stealing algorithm internally.

Q26. How do you gracefully shut down a producer-consumer system with multiple threads?

Strategy 1 (Poison pill): producer enqueues N poison pills (one per consumer). Consumers exit on receiving poison pill. Strategy 2 (interrupt): set a volatile boolean `running = false`, interrupt all consumer threads so they exit from `take()`'s blocking. Strategy 3 (`ExecutorService`): `executor.shutdown() + awaitTermination()`.

Q27. What is the happens-before guarantee provided by ArrayBlockingQueue.put() -> take()?

Per JMM: `put(e)` in Thread A happens-before `take()` in Thread B returns `e`. This means all writes in Thread A BEFORE `put()` are visible to Thread B AFTER `take()`. No additional synchronization needed between producer and consumer when using `BlockingQueue`.

Summary Chapter Summary & Quick Reference

This chapter covered Java's Queue, Deque, and BlockingQueue hierarchy in depth. PriorityQueue uses a binary min-heap ($O(\log n)$ offer/poll, $O(1)$ peek) — not a sorted collection. ArrayDeque is the preferred single-threaded stack and queue, using a circular array with $O(1)$ amortized operations and no null support. For concurrent producer-consumer: ArrayBlockingQueue (bounded, one lock, back-pressure) or LinkedBlockingQueue (optional bounds, two locks, higher throughput). SynchronousQueue is the zero-capacity handoff queue used by newCachedThreadPool(). DelayQueue enables time-based scheduling via the Delayed interface. PriorityBlockingQueue is the thread-safe but unbounded priority queue. The poison pill pattern and executor shutdown() are the standard graceful termination strategies.

TIP: Decision guide: single-threaded FIFO -> ArrayDeque. Single-threaded priority -> PriorityQueue. Multi-threaded bounded buffer -> ArrayBlockingQueue. Multi-threaded unbounded -> LinkedBlockingQueue. Thread handoff -> SynchronousQueue. Timed scheduling -> DelayQueue.

Use Case	Recommended Class
Single-threaded FIFO queue	ArrayDeque (as Queue)
Single-threaded LIFO stack	ArrayDeque (push/pop)
Single-threaded priority scheduling	PriorityQueue with Comparator
Bounded producer-consumer	ArrayBlockingQueue
Unbounded producer-consumer	LinkedBlockingQueue
Concurrent priority scheduling	PriorityBlockingQueue
Thread-to-thread handoff (no buffering)	SynchronousQueue
Time-delayed task scheduling	DelayQueue
Work stealing (concurrent deque)	LinkedBlockingDeque
High-throughput async processing	LinkedTransferQueue